

**SIMATS ENGINEERING**  
**ASSESSMENT TOOL 3: Reverse Engineering Task**

**COURSE NAME:** Cloud Computing and  
Big Data Analytics

**COURSE CODE:** CSA1522

---

**COURSE OUTCOME COVERED**

**CO4:** *Analyze big data characteristics and challenges across domains including security and application aspects.*  
(BL4)

Dave's Taxonomy: Articulation (Level 4)  
Question Count: 5

**Total Marks: 30**

---

**Code of Conduct**

I Kanmani L/192511414 (Name / Reg No) certify that this submission is my original work  
and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: \_\_\_\_\_



## Assessment Tasks

### Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

**Reverse Engineering Task Rubric**

| <b>Criteria</b>               | <b>Weightage</b> | <b>Level 4 - Exemplary</b>                                    | <b>Level 3 - Proficient</b> | <b>Level 2 - Developing</b> | <b>Level 1 - Beginning</b> |
|-------------------------------|------------------|---------------------------------------------------------------|-----------------------------|-----------------------------|----------------------------|
| Inference Accuracy            | 25%              | Infers system components and flow with high accuracy          | Mostly accurate inference   | Basic inference             | Weak inference             |
| Understanding of Architecture | 25%              | Shows clear understanding of underlying big data architecture | Good understanding          | Partial understanding       | Poor understanding         |
| Use of Technical Evidence     | 20%              | Uses strong reasoning and technical evidence                  | Reasonable evidence         | Limited evidence            | No meaningful evidence     |
| Depth of Analysis             | 15%              | Analysis is deep and insightful                               | Reasonably deep             | Basic depth                 | Superficial analysis       |
| Clarity                       | 15%              | Clear and organized explanation                               | Generally clear             | Somewhat unclear            | Poorly presented           |

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

Answer:

A big data platform handling billions of website clicks, mobile interactions, and customer transactions would most likely use a distributed architecture such as Hadoop and cloud-based big data technologies.

#### Distributed Storage System

A platform handling billions of records would likely use HDFS (Hadoop Distributed File System) or cloud-based data lakes such as Amazon S3. Data is distributed across multiple machines, allowing large volumes of data to be stored efficiently. Replication provides fault tolerance if a node fails.

#### Data Ingestion

Apache Kafka would be a suitable ingestion framework for collecting website clicks, mobile events, and transaction records. Kafka can handle high-speed data streams and distribute incoming data across multiple partitions.

#### Batch Processing

For processing large volumes of historical data, Apache Spark or Hadoop MapReduce could be used. Spark is preferred for faster processing and supports SQL, machine learning, and large-scale analytics.

#### Data Organization

Structured data such as customer and transaction records can be stored in relational or distributed databases. Semi-structured data such as JSON logs, clickstreams, and mobile events can be stored in a data lake using formats such as JSON, Avro, or Parquet. Data can be partitioned by date, customer, region, or event type for faster querying.

#### Real-Time Processing

For real-time analytics, technologies such as Apache Spark Streaming, Apache Flink, or Kafka Streams can process events as they arrive. They can support real-time applications such as transaction monitoring, customer activity tracking, and fraud detection.

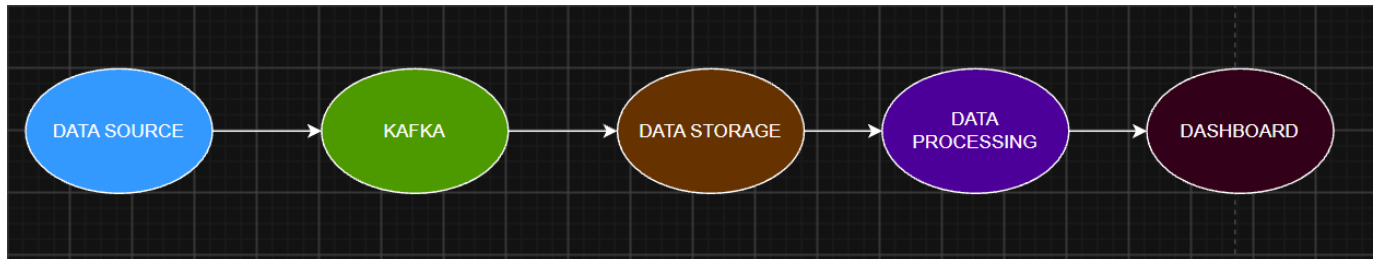
#### Scalability and Partitioning

The system would use horizontal scaling, where additional servers are added when the workload increases. Data is divided into partitions and processed in parallel across multiple nodes. Kafka partitions and distributed storage help balance the workload and prevent a single server from becoming a bottleneck.

## Dashboards and Business Intelligence

After processing, the data can be stored in a data warehouse or analytics layer. Business Intelligence tools such as Power BI, Tableau, or Looker can connect to this layer to create dashboards and reports for business users. These dashboards can display customer behavior, sales trends, transactions, and real-time performance.

Architecture:



2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

Answer:

An e-commerce platform generates huge amounts of data from product views, cart additions, purchases, payments, and customer interactions. Since this data is generated continuously across different regions, a big data architecture is required to collect, process, store, and analyze it. The platform can use distributed technologies for real-time personalization, recommendation, analytics, and KPI monitoring.

### Event Streaming Tools

The platform would likely use Apache Kafka to collect events such as product views, cart additions, abandoned checkouts, and payments. Kafka can handle millions of events and distribute them across multiple partitions. Kafka Streams or Apache Flink can process these events in real time.

### Customer Profiling Database

Customer profiles containing customer ID, preferences, purchase history, and browsing behavior could be stored in databases such as MongoDB, Cassandra, or Redis. Redis can also be used for frequently accessed customer information and real-time personalization.

## Recommendation Workflow

The recommendation system would collect browsing and transaction events, process them using Apache Spark or Flink, and generate customer features such as frequently viewed products, preferred categories, and purchase patterns. Machine learning models can then recommend relevant products to customers.

## Real-Time Data Synchronization

Session data, clickstream logs, and transaction records can be synchronized using Kafka. Each event contains information such as customer ID, session ID, timestamp, and event type. Stream-processing tools combine these events to maintain an updated view of customer activity and provide real-time recommendations.

## Machine Learning Stages

The machine learning workflow includes data cleaning, feature extraction, model training, prediction, and recommendation. Features may include product views, cart activity, previous purchases, browsing frequency, and customer preferences. The trained model predicts products that a customer is likely to purchase.

## Merging Structured and Semi-Structured Data

Structured order and payment data can come from relational databases such as MySQL or PostgreSQL. Semi-structured browsing data such as JSON clickstream logs can be stored in a data lake. Both can be joined using common fields such as customer ID, product ID, and session ID for combined analytics.

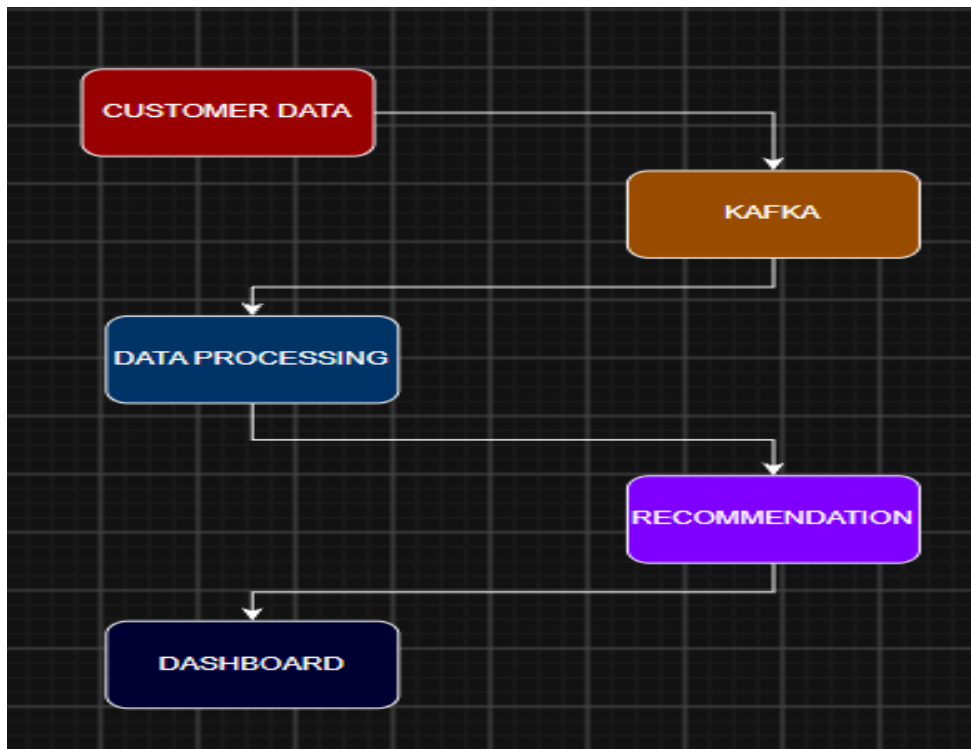
## Scalability and Fault Tolerance

The platform would use horizontal scaling by adding more servers as traffic increases. Kafka partitions distribute events across multiple brokers, while Spark or Flink distributes processing across nodes. Data replication and checkpointing help recover from failures and prevent data loss.

## Dashboards and KPI Monitoring

Processed data can be stored in a data warehouse or analytics layer and connected to Power BI, Tableau, or Looker. Dashboards can monitor KPIs such as conversion rate, cart abandonment rate, product views, regional sales, average order value, and recommendation performance.

Architecture:



3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

Answer:

A healthcare system generates large volumes of structured, semi-structured, and unstructured data such as patient records, diagnostic images, wearable sensor readings, and prescription details. A big data platform is required to integrate these different data sources, process them securely, and provide useful insights for disease prediction, patient monitoring, and clinical decision-making.

Hybrid Database Architecture

A hybrid database architecture can be used to manage different types of healthcare data. Structured data such as patient details, prescriptions, and billing records can be stored in MySQL, PostgreSQL, or Oracle. Unstructured data such as X-rays, CT scans, MRI images, and medical documents can be stored in a distributed file system or cloud data lake. MongoDB or another NoSQL database can be used for semi-structured records.

## Data Integration

Hospitals, laboratories, and insurance systems can exchange data using integration tools such as Apache Kafka, Apache NiFi, or APIs. Healthcare standards such as HL7 and FHIR can help different systems exchange patient information in a consistent format. This allows laboratory results, hospital records, prescriptions, and insurance information to be combined.

## Streaming and Machine Learning

Wearable devices continuously generate heart rate, blood pressure, glucose, and activity data. Apache Kafka can collect these streams, while Apache Spark Streaming or Apache Flink can process them in real time. Machine learning frameworks such as TensorFlow, PyTorch, or Spark MLlib can be used for disease prediction, risk analysis, medical image analysis, and patient monitoring.

## Privacy and Security

Healthcare data requires strong security because it contains sensitive patient information. Encryption at rest and in transit can protect stored and transmitted data. Role-Based Access Control (RBAC), authentication, audit logs, and data masking can restrict unauthorized access. The platform would also follow applicable healthcare privacy and security regulations, such as HIPAA where applicable, along with relevant local data-protection requirements.

## Analytics Pipeline

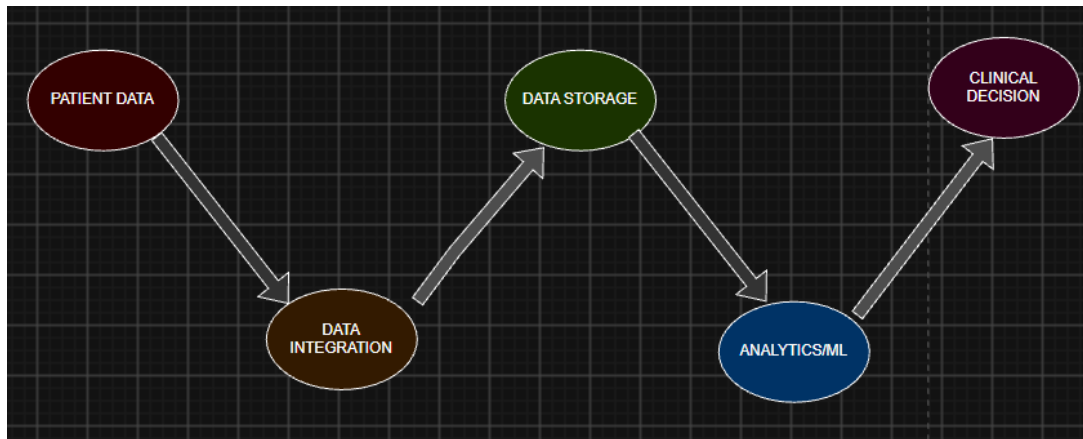
The analytics pipeline can collect data from hospitals, laboratories, wearable devices, and insurance systems. The data is then ingested, cleaned, integrated, stored, and processed using big data technologies. Machine learning models analyze the data to identify disease risks, predict patient outcomes, and detect abnormal conditions.

## Clinical and Population Health Analytics

Processed information can be presented through Power BI, Tableau, or healthcare dashboards. Doctors can use the results for patient-level decision-making, while healthcare organizations can analyze large populations to identify disease trends, high-risk groups, treatment outcomes, and healthcare resource requirements. This supports both clinical decision-making and population health studies.



Architecture:



4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

Answer;

A smart city generates continuous data from traffic sensors, CCTV systems, weather stations, and public transport systems. A big data platform is required to process this data in real time and support traffic management, congestion prediction, urban planning, and public safety.

#### Edge Computing

Edge devices or gateways can process sensor and CCTV data close to where it is generated. This reduces network traffic and provides faster responses for applications such as traffic signal control and congestion detection.

#### Distributed Messaging and Storage

Apache Kafka can be used as the distributed messaging system to collect continuous data from different city services. A centralized data lake using HDFS or cloud storage can store large volumes of historical sensor and CCTV data.

#### Geospatial and Streaming Analytics

Apache Spark Streaming or Apache Flink can process traffic and transport data in real time. Geospatial analytics can identify vehicle locations, traffic density, and congested areas. Historical traffic patterns can then be combined with live data to predict congestion.

## Databases

InfluxDB or TimescaleDB can store time-series sensor data efficiently. PostGIS with PostgreSQL can be used for location-based queries such as finding traffic conditions within a particular area or route.

## Batch and Real-Time Analytics

Real-time processing provides immediate information about traffic congestion, accidents, and transport delays. Batch processing using Spark can analyze historical data to identify long-term traffic patterns and support urban planning decisions.

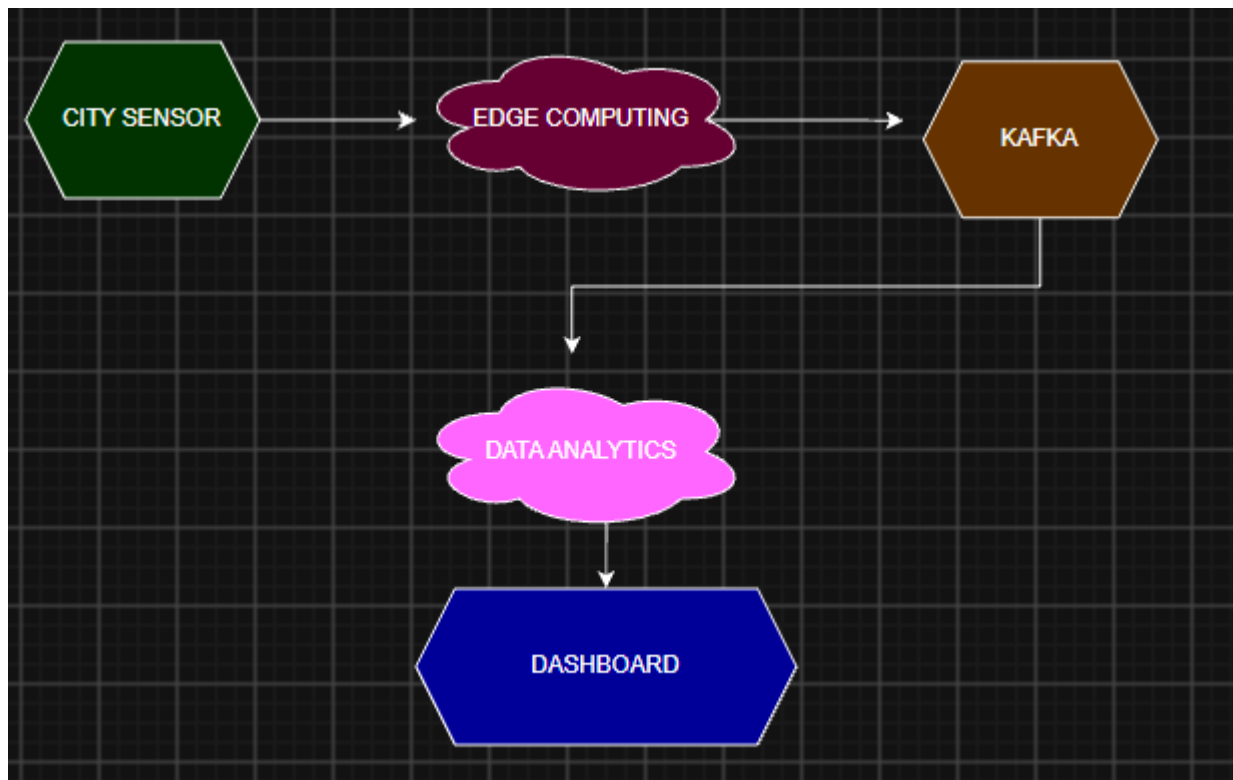
## Security and Access Control

Citizen and infrastructure data can be protected using encryption, authentication, role-based access control, and secure APIs. Access can be restricted so that only authorized departments can view or modify specific types of data. Audit logs can also track data access.

## Visualization and Operational Intelligence

Tools such as Power BI, Tableau, or Grafana can display live dashboards showing traffic density, congestion areas, weather conditions, and public transport status. This provides city authorities with real-time information for faster decision-making.

## Architecture:



5.A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Answer:

A healthcare analytics system processes data from patient records, wearable devices, laboratories, and appointment systems. Since healthcare data is large and sensitive, a big data architecture is required for secure storage, real-time monitoring, predictive analytics, and clinical decision-making.

#### Hybrid Storage Architecture

Structured data such as patient details, appointments, and lab results can be stored in MySQL or PostgreSQL. Unstructured data such as medical reports and images can be stored in a cloud data lake or HDFS. NoSQL databases such as MongoDB can manage semi-structured wearable and medical data.

#### ETL and Interoperability

Apache NiFi or Talend can be used for ETL to extract, clean, transform, and load data from different healthcare systems. HL7 and FHIR standards can help hospitals and clinics exchange patient information in a common format.

#### Real-Time and Predictive Analytics

Wearable devices continuously send readings such as heart rate and blood pressure. Kafka can collect these streams, while Spark Streaming or Flink can analyze them in real time. Abnormal readings can trigger alerts, while predictive models can identify potential health risks.

#### Distributed Processing

Apache Spark can process large datasets across multiple machines. This enables population-level analysis of diseases, patient trends, treatment outcomes, and risk factors much faster than processing the entire dataset on a single system.

#### Security and Data Governance

Sensitive medical information can be protected using encryption, authentication, role-based access control, and audit logs. Compliance requirements such as HIPAA where applicable can be supported through controlled access, data protection, and monitoring of data usage.

## Machine Learning

Machine learning models can analyze patient history, laboratory results, and wearable readings to calculate disease-risk scores. Models can also help identify high-risk patients and support doctors in selecting suitable treatments based on patient data and previous outcomes.

Architecture:

